SMART HEALTH AND SAFETY MONITORING SYSTEM

Project type: Arduino-based embedded monitoring prototype **Platform:** Arduino UNO + Python

This document describes the project for a public GitHub repository. It focuses on the system design, components, setup, data flow, testing, limitations, and future improvements. Source-code listings have intentionally been excluded; the repository should contain the Arduino sketch and Python dashboard as separate files.

Project Overview

The Smart Health and Safety Monitoring System is an educational prototype that combines environmental sensing, GPS location data, a local OLED display, Bluetooth serial communication, and a computer dashboard. An Arduino UNO reads sensor modules, formats readings into a serial record, updates the display, and sends data to a Python application. The dashboard presents live readings, plots recent values, and can save records for later review.

The system is intended for learning and demonstration. It is not a certified medical device, industrial gas detector, or emergency-response system. The MQ-2 sensor provides a raw, environment-dependent response and must not be treated as a calibrated gas concentration measurement.

Features

- Reads temperature and relative humidity from a DHT11 sensor.
- Reads the MQ-2 analogue output as a raw gas/smoke indicator and applies configurable demonstration thresholds.
- Receives latitude, longitude, satellite count, and GPS-fix status from a NEO-6M receiver when reception is available.
- Displays selected readings and status messages on a 0.96-inch I2C OLED.
- Transmits structured data to a paired computer using an HC-05 Bluetooth serial link.
- Provides a Python desktop dashboard with live values, trend plots, GPS information, and Excel logging.

1. Introduction

Environmental monitoring prototypes help demonstrate how sensors, microcontrollers, displays, and communication interfaces can work together. In this project, the Arduino UNO serves as the central controller. The OLED provides immediate local feedback, while the Bluetooth serial connection sends data to a nearby computer for visualization and logging.

The project brings together embedded programming, sensor interfacing, serial communication, data parsing, plotting, and basic data persistence. Its modular structure also makes it suitable for testing individual components before combining them.

2. Problem Definition

Checking several sensor readings independently and recording them manually makes it difficult to observe trends or demonstrate a complete monitoring workflow. The project addresses this by collecting selected readings in one prototype, presenting them locally, and transferring structured data to a computer.

Design requirements

- Acquire temperature and humidity readings from the DHT11.
- Read the MQ-2 analogue signal and display a threshold-based status label.
- Report GPS coordinates when a valid fix is available and indicate when no fix exists.
- Refresh the OLED with readable sensor values and status information.
- Send consistently formatted records over Bluetooth serial.
- Parse incoming records in Python and support plotting and spreadsheet logging.

3. System Architecture

The workflow is divided into five blocks: sensing, processing, local display, wireless communication, and PC-side monitoring. The DHT11, MQ-2, and NEO-6M provide input data. The Arduino processes these inputs and updates the OLED. A Bluetooth serial link forwards records to a paired computer, where the Python dashboard parses, displays, plots, and stores valid samples.

Block	Main role
Sensors	Collect temperature, humidity, raw gas-sensor response, and GPS information.
Arduino UNO	Coordinates sensor reading, data formatting, display refresh, and transmission.
OLED display	Shows selected readings and local status.
HC-05 Bluetooth	Transfers serial records to a paired PC; it does not itself provide internet access.
Python dashboard	Receives records, updates values and plots, shows GPS state, and logs samples.

4. Hardware Components

Component	Function and notes
Arduino UNO	Central controller for input acquisition and output handling.
DHT11	Basic digital temperature and relative-humidity readings.
MQ-2 module	Analogue response to smoke and certain combustible gases; requires warm-up and appropriate calibration for meaningful measurement.
NEO-6M GPS	Provides location and satellite information when signal conditions permit.
0.96-inch I2C OLED	Displays local readings; commonly uses I2C address 0x3C, which should be verified.
HC-05 Bluetooth	Provides a Bluetooth serial connection to a paired computer.
Breadboard, jumper wires, USB/power supply	Used for prototype assembly, programming, and power.

5. Software and Libraries

Tool / library	Purpose
Arduino IDE	Edit, compile, and upload the firmware.
DHT sensor library	Read DHT11 measurements.
Adafruit SSD1306 and Adafruit GFX	Render text and graphics on the OLED.
TinyGPSPlus	Parse GPS receiver messages.
SoftwareSerial	Provide serial interfaces on selected Arduino pins; simultaneous listening on multiple ports is limited on many AVR boards.
Python 3 and pySerial	Read data from the Bluetooth serial port.
Matplotlib	Plot recent temperature, humidity, and raw gas-sensor readings.
openpyxl	Create or update an Excel workbook for recorded samples.

6. Wiring Reference

Module / signal	Arduino UNO connection	Notes
DHT11 DATA	D7 in the representative firmware	Use the module's pull-up arrangement or the recommended pull-up for a bare sensor.
MQ-2 analogue output	A0	Allow warm-up; raw ADC values are not ppm.
OLED SDA	A4 / SDA	Confirm the display address and voltage requirements.
OLED SCL	A5 / SCL	Confirm module pin labels.
GPS TX	D8 (Arduino GPS RX)	Matches the representative firmware pin definitions.
GPS RX	D9 (Arduino GPS TX)	May be unnecessary if only receiving GPS data; check logic levels.
HC-05 TXD	D10 (Arduino BT RX)	Arduino receives module output.
HC-05 RXD	D11 (Arduino BT TX)	Use suitable level shifting or a voltage divider where required.
Grounds	Common GND	A shared reference is required for reliable signals.

Always check the exact pin definitions in the firmware committed to the repository before wiring. Confirm the voltage limits of each module and disconnect power before changing connections.

7. Data Format and Dashboard

The representative firmware sends one comma-separated line beginning with the marker **$D,**. The expected field order is:

$D,temp,hum,gas,lat,lon,sats,fix

The fields represent temperature, relative humidity, raw MQ-2 reading, latitude, longitude, satellite count, and GPS-fix flag. The dashboard should accept only complete records with the expected number of fields, handle malformed input without crashing, label units clearly, and timestamp logged samples. When there is no valid GPS fix, the dashboard should indicate that the location is unavailable rather than presenting placeholder coordinates as a real position.

The firmware and dashboard must use the same baud rate and data format. The representative settings use 9600 baud. Select the incoming Bluetooth COM port shown by the operating system and close any other program that is using it before connecting.

8. Installation and Usage

Hardware preparation

- Assemble the modules according to the wiring reference and verify all power and ground connections.
- Install the Arduino IDE and the required libraries: DHT sensor library, Adafruit SSD1306, Adafruit GFX, and TinyGPSPlus.
- Open the Arduino sketch, confirm the pin definitions and OLED configuration, compile it, and upload it to the board.
- Allow the MQ-2 sensor to warm up and test GPS outdoors with the antenna in a suitable position.

Python dashboard preparation

- Install Python 3.
- Install the dashboard dependencies: pyserial, matplotlib, and openpyxl.
- Pair the computer with the HC-05 and identify the incoming Bluetooth serial port.
- Run the dashboard, select the correct port, and connect.
- Confirm that readings update and that valid samples are logged to the configured workbook.

Example dependency installation command: `pip install pyserial matplotlib openpyxl`

9. Testing and Results

The original report describes functional checks and expected observations rather than verified measured results. The following checklist should be completed with the actual readings, screenshots, and test conditions from the assembled prototype before publishing the repository.

Test	Procedure	Expected observation
OLED refresh	Power the system and observe the display.	Text appears and the screen refreshes consistently.
DHT11	Compare multiple readings over time.	Temperature and humidity values update; invalid readings are handled.
MQ-2 response	Allow warm-up and observe the raw analogue value under controlled conditions.	Raw reading changes with conditions and the configured label follows the selected threshold.
GPS	Test outdoors with antenna facing the sky.	Coordinates and fix status appear after satellites are acquired; otherwise no-fix status is shown.
Bluetooth	Pair the HC-05 and open its incoming serial port.	Formatted records arrive at the configured baud rate.

Test	Procedure	Expected observation
Dashboard	Start the Python program and transmit multiple records.	Values and plots update; malformed lines do not crash the application.
Logging	Enable data logging and inspect the output workbook.	Timestamped samples are saved and can be opened in spreadsheet software.

10. Limitations and Safety Notes

- **MQ-2:** The raw analogue response is not a calibrated concentration measurement. Warm-up, sensor resistance, humidity, supply conditions, placement, and gas mixture affect readings.
- **DHT11:** Intended for basic environmental observation rather than precision measurement.
- **GPS:** Indoor use and obstructions can prevent a valid fix.
- **Bluetooth:** Requires pairing and remains limited by radio range and connection conditions.
- **Prototype wiring:** Breadboards and jumper wires can loosen; a soldered PCB and protected enclosure would be more robust.
- **Safety:** Threshold labels such as Normal, Elevated, or HIGH are demonstration indicators only. Do not rely on this project to protect people, certify an environment as safe, or replace approved alarms and professional safety equipment.

11. Future Improvements

- Create a compact custom PCB and durable enclosure with suitable sensor openings.
- Improve serial-port reconnection, input validation, error messages, and configurable thresholds.
- Use calibrated sensors and certified alarms for any real safety-critical application.
- Improve battery management and include battery-status monitoring for portable operation.
- Add exportable reports, more robust storage, and optional network synchronization.
- Measure and document response time, repeatability, communication range, and battery runtime.
- Improve the dashboard with clearer units, timestamps, connection state, and accessible status messages.

References

1. Arduino, “Arduino UNO Rev3,” https://docs.arduino.cc/hardware/uno-rev3/
2. Arduino, “SoftwareSerial Library,” https://docs.arduino.cc/
3. Adafruit, “Adafruit SSD1306 Library,” https://github.com/adafruit/Adafruit_SSD1306
4. Adafruit, “Adafruit GFX Library,” https://github.com/adafruit/Adafruit-GFX-Library
5. Adafruit, “DHT sensor library,” https://github.com/adafruit/DHT-sensor-library

6. Mikal Hart, “TinyGPSPlus,” https://github.com/mikalhart/TinyGPSPlus

7. Python Software Foundation, “Python Documentation,” https://docs.python.org/3/

8. pySerial, “pySerial API,” https://pyserial.readthedocs.io/

9. Matplotlib, “Matplotlib Documentation,” https://matplotlib.org/stable/

10. openpyxl, “openpyxl Documentation,” https://openpyxl.readthedocs.io/

11. MQ-2 gas sensor module: consult the datasheet for the exact module purchased.

12. u-blox, “NEO-6 GPS Modules Data Sheet”: consult the revision matching the module used.

Project team: A. Manohar Reddy, D. Parthav Kumar, P. Raja Simha Reddy, S. Yeswanth Reddy, P. Hemanth Reddy, and G. Sushanth.

Institution: VIT-AP University, Amaravati, Andhra Pradesh, India. Academic Year: 2026–2027. Guide: Prof. Dr. Raja Solanki.